

Sum of Digits

Sum of Digits

Given a **positive** integer N , design an algorithm to find the **sum** of its **digits**.

Example

Input : $N = 123$
Output : 6

$1 + 2 + 3 = 6$

$sum = \cancel{0} \cancel{3} \cancel{5} 6$

Input : $N = 456$
Output : 15

$4 + 5 + 6 = 15$

$1 = \cancel{4} \cancel{5} \cancel{6}$
 $\quad \quad \quad \underline{45}$
 $\quad \quad \quad \underline{4}$
 $\quad \quad \quad 0$

$sum = \cancel{0} \cancel{6} \cancel{4} 4$

Reverse a number

Reverse a Number

Given a **positive** number **N**, design an algorithm to reverse the number.

Example

Input : N = 123

Output : 321

Input : N = 456

Output : 654

$$\begin{matrix} (1 & 0 & 1 & 0 & 1)_2 & \rightarrow & (21)_{10} \\ 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \end{matrix}$$

$$16 + 0 + 4 + 0 + 1$$

$$\begin{matrix} \downarrow & \longrightarrow \\ (1 & 0 & 1 & 0 & 1)_2 & \rightarrow & ()_{10} \end{matrix}$$

$$\begin{matrix} \leftarrow \\ (123)_{10} \rightarrow (?)_{10} \end{matrix}$$

$$\begin{aligned} &0 \times 10 + 1 \\ &= 1 \times 10 + 2 \\ &= 12 \times 10 + 3 \\ &= 123 \end{aligned}$$

$$\begin{aligned} &0 \times 2 + 1 \\ &= 1 \times 2 + 0 \\ &= 2 \times 2 + 1 \\ &= 5 \times 2 + 0 \\ &= 10 \times 2 + 1 \\ &= 21 \end{aligned}$$

$$\begin{aligned} &0 \times 10 + 3 \\ &= 3 \times 10 + 2 \\ &= 32 \times 10 + 1 \\ &= \underline{\underline{321}} \end{aligned}$$

i/p

$$n = \cancel{456}$$

o/p

$$\underline{\underline{654}}$$

ans = ~~0665~~ 654

$$\begin{aligned} &0 * 10 + 6 \\ &6 * 10 + 5 \\ &65 * 10 + 4 \\ &654 \end{aligned}$$

Variable Scope

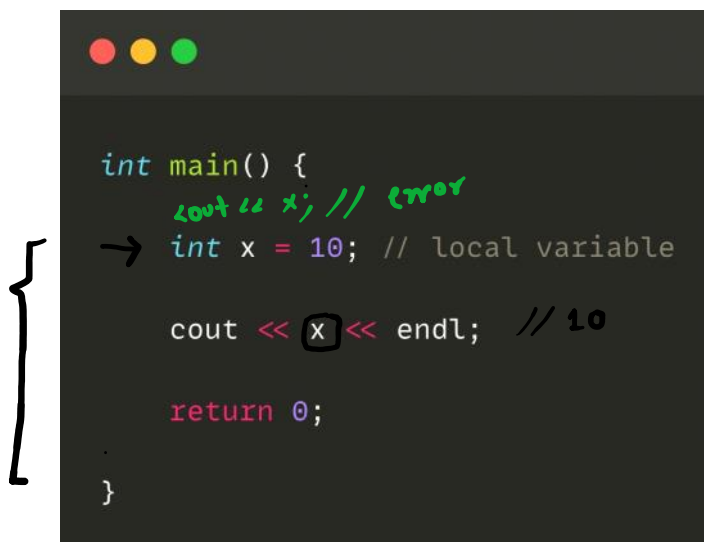
In C++, **scope** of a variable can be thought of as the **visibility** of the variable. In general, a variable can either have a **block scope** or a **global scope**.

- **Block Scope**

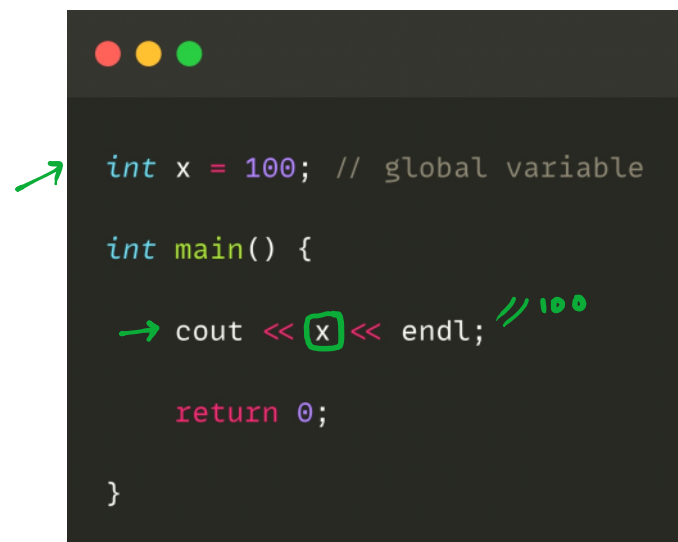
⇒ Any variable declared within a block (`{ }`) has a **block scope** i.e. it is visible / accessible from the point of its declaration till the end of the block. We call variables with block scope as **local variables**.

- **Global Scope**

Any variable declared not within a block (`{ }`) has a **global scope** and we call such variables as **global variables**.



```
int main() {  
    cout << x; // error  
    → int x = 10; // local variable  
    cout << x << endl; // 10  
    return 0;  
}
```



```
int x = 100; // global variable  
int main() {  
    → cout << x << endl; // 100  
    return 0;  
}
```

Variable Scope Resolution

```

int main() {
    int x = 10;
    cout << x << endl; // 10

    {
        int y = 20;
        cout << x << " " << y << endl; // 10 20
    }
    cout << x; // 10
    return 0;
    cout << y; // error
}

```

```

int main() {
    int x = 10;
    cout << x << endl; // 10

    {
        int x = 30;
        int y = 20;
        cout << x << " " << y << endl; // 30 20
    }
    cout << x; // 10
    return 0;
}

```

```

int x = 100; // global variable

int main() {
    cout << x << endl; // 100

    int x = 10;
    cout << x << endl; // 10

    {
        int x = 30;
        int y = 20;
        cout << x << " " << y << endl;
    }
    cout << x; // 10
    return 0;
}

```

30 20

Scope Resolution Operator [::]

One of the use cases of the scope resolution operator [::] is to be able to **access** a global variable when there is a local variable with the same name.

```

int x = 100; // global variable

int main() {
    int x = 10; // local variable

    cout << x << " " << ::x << endl;
    ::x++;
    cout << x << " " << ::x << endl;

    return 0;
}

```

10 100

10 101

::x = ::x + 1;

Static Variables

local var.

global var.

1. block scope

1. global scope

2. automatic
= duration

2. static duration

static var. { block scope
static duration

{

int x;

static int y;

}

visibility is w/i the block
deallocated as soon as
block gets over

visibility is w/i the block ✓
remains in the mem. as
long as prog. runs. ✓

by default static/global
they contain zero

static var. are init } *
only once

Fibonacci Numbers

 ≥ 0 Given a non-negative integer n , design an algorithm to find the n^{th} Fibonacci number.

$$f_6 = f_5 + f_4 = 5 + 3 = 8$$

 $\eta = 4$
 $o/p: 3$

n	0	1	2	3	4	5	6	7	8	9	...
f(n)	0	1	1	2	3	5	8	13	21	34	...

 $\eta = 8$
 $o/p: 21$

← fib. seq.

$$f_0 \rightarrow 0$$

$$f_1 \rightarrow 1$$

$$f_n = f_{n-1} + f_{n-2}$$

$$\eta \geq 2$$

if ($n == 0$ || $n == 1$) {

 cout << n;

} else {

 $\eta = 2$
 $a = 0$ // 0th
 $b = 1$ // 1st

{

$c = a + b$; // 2nd

$a = b$;

$b = c$;

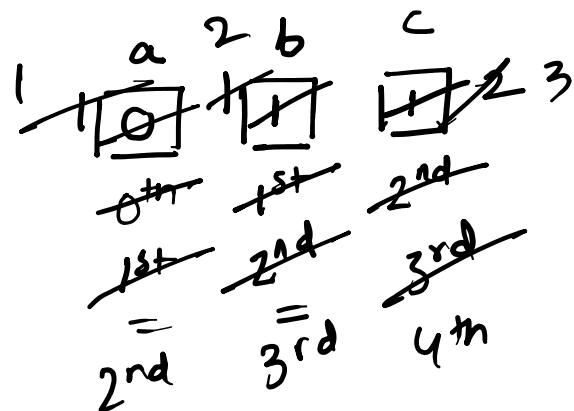
{

$c = a + b$; // 3rd

$a = b$;

$b = c$;

$c = a + b$; // 4th



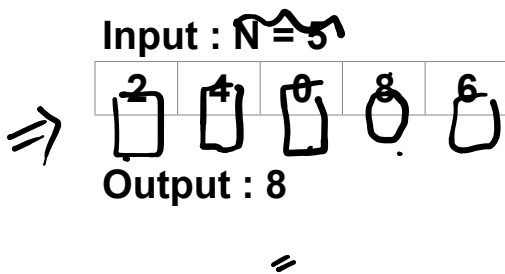
```
L  b = c;  
{  c = a + b; // 4th  
}
```


Largest of N numbers

Largest of N numbers

Given **N** integer values, design an algorithm to find the **largest** of the **N** numbers.

Example



Isf = - ~~90~~ / ~~2~~ ~~4~~ 8 =

Controlling Loops in C++

- **Break Statement**

The break statement is used to **pre-maturely terminate** a loop.

```
int main() {
    int count = 0;
    int data;
    while(true) {
        cin >> data;
        if(data < 0) {
            break;
        }
        count++;
    }
    cout << count << endl;
    return 0;
}
```

while (....) {

while (....) {

break;

}

}

you break
out of
the inner
while loop

- **Continue Statement**

The continue statement is used to **skip** an iteration of the loop.

0 < 5? ✓
1 < 5? ✓
2 < 5? ✓
3 < 5? ✓
4 < 5? ✓
5 < 5? X
1 == 2? X
2 == 2? ✓
3 == 2? X
4 == 2? X
5 == 2? X

```
int main() {
    int i = 0;
    → while(i < 5) {
        → i = i+1;
        → if(i == 2) {
            → continue;
        }
        cout << i << " ";
    }
    return 0;
}
```

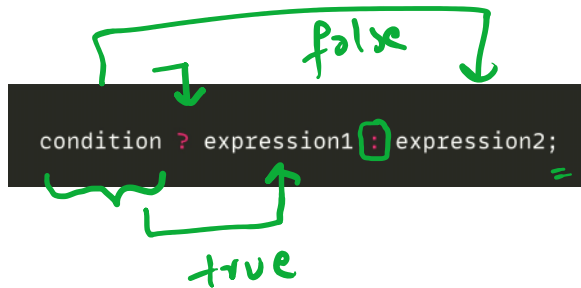
i = 0, 1, 2, 3, 4, 5

o/p: 1, 3, 4, 5

Ternary Operator in C++



The **ternary** or **conditional** operator **evaluates** a condition & **executes** a block of code based on the result of the condition.



```
int main() {  
    int marks;  
    cin >> marks;  
    if(marks > 40) {  
        cout << "pass";  
    } else {  
        cout << "fail";  
    }  
    return 0;  
}
```

```
int main() {  
    int marks;  
    cin >> marks;  
    marks > 40 ? cout << "pass" : cout << "fail";  
    return 0;  
}
```

```

int main() {
    int i;
    cin >> i;
    int j;
    if(i%2 == 0) {
        j = 1;
    } else {
        j = 0;
    }
    return 0;
}

```

```

int main() {
    int i;
    cin >> i;
    int j = i%2 == 0 ? 1 : 0;
    return 0;
}

```

Handwritten annotations in green: A bracket groups the entire ternary expression. An arrow points from the text "true" to the condition `i%2 == 0`. Another arrow points from the text "false" to the colon and the value `0`.

In C++, the ternary or conditional operator can be **nested** i.e. we can use a ternary operator within a ternary operator.

```

int main() {
    int n;
    cin >> n;
    if(n > 0) {
        cout << "+ive";
    } else if(n < 0) {
        cout << "-ive";
    } else {
        cout << "zero";
    }
    return 0;
}

```

```

int main() {
    int n;
    cin >> n;
    n > 0 ? cout << "+ive" :
    n < 0 ? cout << "-ive" : cout << "zero";
    return 0;
}

```

Handwritten annotations in green: Brackets group the first and second ternary expressions. A circle highlights the colon in the first ternary expression. The text "zero" in the final output is underlined.

Check Fibonacci Number

Given a non-negative integer **N**, check if it is a **Fibonacci** number.

n	0	1	2	3	4	5	6	7	8	9	...
f(n)	0	1	1	2	3	5	8	13	21	34	...

Example :

Input : N = 10

Output : false

Input : N = 8

Output : true

Check Prime

Check Prime

Given a positive number **N** (**>1**), design an algorithm to **check** if the given number is a **prime** number.

Example

Input : N = 5

Output : true



Input : N = 4

Output : false

2 3 4
→

$\eta = 9$

7
↓
2 3 4 5 6 7 8
→

A number **n** is a **prime no.** if it is **divisible** by only **1** & the **number** itself.

Square Root of a Number

Find Square Root

Given a non-negative integer **N**, design an algorithm to find its **square root**.

Example

Input : N = 64

Output : 8

Input : N = 30

Output : 5.47

$$1. \sqrt{n} \geq 0 \quad 2. \left| (\sqrt{n})^2 \leq n \right| \Leftrightarrow$$

$$\sqrt{0} = 0 \quad \sqrt{25} = 5$$

$$\sqrt{1} = 1 \quad \sqrt{36} = 6$$

$$\sqrt{4} = 2 \quad \sqrt{49} = 7$$

$$\sqrt{9} = 3 \quad \sqrt{64} = 8$$

$$\sqrt{16} = 4 \quad \sqrt{81} = 9$$

$$\cdot 0^2 \leq 64$$

$$\cdot 1^2$$

$$2^2$$

$$\vdots$$

$$7^2$$

$$\cdot 8^2$$

$$\sqrt{25}$$

$$0^2 \leq 25? \checkmark$$

$$1^2 \leq 25? \checkmark$$

$$2^2 \leq 25? \checkmark$$

$$3^2 \leq 25? \checkmark$$

$$4^2 \leq 25? \checkmark$$

$$5^2 \leq 25? \checkmark$$

$$6^2 \leq 25? \times$$

$$7 = 42$$

$$6 \cdot 0^2 \leq 42? \checkmark$$

$$6 \cdot 1^2 \leq 42? \checkmark$$

$$\text{ans} = 6$$

$$\vdots$$
$$\underline{\underline{6.4}}^2 \leq 42? \checkmark$$
$$\underline{\underline{6.5}}^2 \leq 42? \times$$